

AQI Islamabad Forecasting System

Internship Report (10Pearls)

Prepared by: Wajiha Babar thaheemwajeeha@gmail.com

Acknowledgements

I am indebted to my mentors and team at 10Pearls for their support and creating the space to work on this tough end-to-end machine learning project. This has provided essential experience in taking theoretical ideas and constructing systems ready for production.

Summary

This report chronicles the end-to-end process of developing an hourly Air Quality Index (AQI) forecasting model for Islamabad. This includes data ingestion, exploratory data analysis, feature engineering, Hopsworks integration (feature store and model registry), model development, SHAP-based model interpretability, a 72-hour inference pipeline, and a Streamlit dashboard.

The final model is a near-real-time forecasting model developed using three models (Random Forest, Ridge Regression, and a Neural Network) and an automated model selection algorithm based on RMSE. The predictions are stored in an online feature group, allowing the dashboard to update without retraining.

Key Results (What the system achieved)

- Hourly feature store created (aqi_features_v2) with ~4k+ rows and clean schema ready for training and online inference.
- Training on last 180 days with time-split approach yielded best results for RandomForest model: RMSE ≈ 19.28 , MAE ≈ 13.93 , $R^2 \approx 0.824$.
- 72-hour inference pipeline produces 72 future timestamps (UTC), makes predictions with 3 models + best RMSE model, and upserts to prediction feature group.
- Streamlit dashboard displays observed history and forecasts, includes model insight cards (RMSEs, best model selection), and export/report functionality.

1. Introduction

The internship project involved developing a comprehensive machine learning forecasting system for predicting the Air Quality Index (AQI) in Islamabad. Rather than considering forecasting as a single notebook task, the project was organized as an engineering pipeline:

- Ingestion of hourly AQI + weather data
- Storage of features in Hopsworks
- Training multiple regression models with version control
- Automated inference for the next 72 hours
- Deployment of the dashboard

The project has developed a functional machine learning system that produces forecasts continuously and provides them through a clean Streamlit dashboard.

2. Problem Statement

The aim of this project is to predict the Air Quality Index (AQI) of Islamabad for the next 3 days (72 hours) based on hourly AQI values and corresponding weather conditions. The forecasting model will utilize:

- AQI patterns
- Weather indicators like temperature, humidity, wind speed, and pressure
- Engineered features based on time series

To make this system feasible and ready for production, it is intended to be an automated pipeline that can:

- Consume new air quality and weather data at any given time
- Store and version features for training and online prediction
- Train, evaluate, and register models with reproducible performance metrics
- Make near-real-time predictions with minimal human effort
- Facilitate model comparison, best-model selection, and prediction storage for future use
- Display forecasts and model insights through a clean and interactive dashboard

The final requirement is to provide an AQI forecasting model that automatically updates its predictions without needing manual retraining every time the dashboard is accessed.

3. Requirements & Constraints

3.1 Functional Requirements

- Retrieve latest AQI and weather information and store as hourly data in a feature store.
- Execute EDA on stored information (trend, seasonality, and correlation with weather variables).
- Train models, test on holdout set based on time, and store versions in Hopsworks Model Registry.
- Automate inference for 72 hours using latest models and weather forecast.
- Provide predictions, alerts, and model comparison on a Streamlit dashboard with export functionality.

3.2 Non-Functional Requirements

- Reproducibility: same code + same feature store snapshot should reproduce metrics.
- Low operational friction: inference should not require retraining each time the app opens.
- Performance: avoid repeated model downloads; keep dashboard refresh predictable.
- Maintainability: clear project structure and isolated pipelines (feature, training, inference, dashboard).

4. Project Structure & Tech Stack

The repository is organized around small, focused pipelines. This helped a lot during debugging because issues stayed localized.

4.1 Repository Structure (high level)

The repository has a modular structure that is engineering-friendly:

- .devcontainer/ → reproducible development environment
- .github/ → GitHub Actions workflow automation
- app/ → Streamlit dashboard frontend
- assets/ → UI images, icons, static resources
- models/ → trained model artifacts
- model_cache/ → cached registry downloads (performance fix)
- notebooks/ → EDA exploration notebook
- outputs/ → exported predictions, plots, logs
- src/ → core ML pipelines (features, training, inference)
- .env → secure API keys and Hopsworks credentials
- requirements.txt → dependency management
- README.md → project documentation

4.2 Tools

- Python (pandas, numpy, scikit-learn, joblib)
- Hopsworks: Feature Store (online + offline) and Model Registry
- Streamlit for dashboard
- OpenWeather + AQICN for current conditions, Open-Meteo for 72h weather forecast

5. System Design Diagrams

The following diagrams summarize how the components interact and how the end-to-end workflow runs.

End-to-End System Flow (AQI Islamabad Forecast)

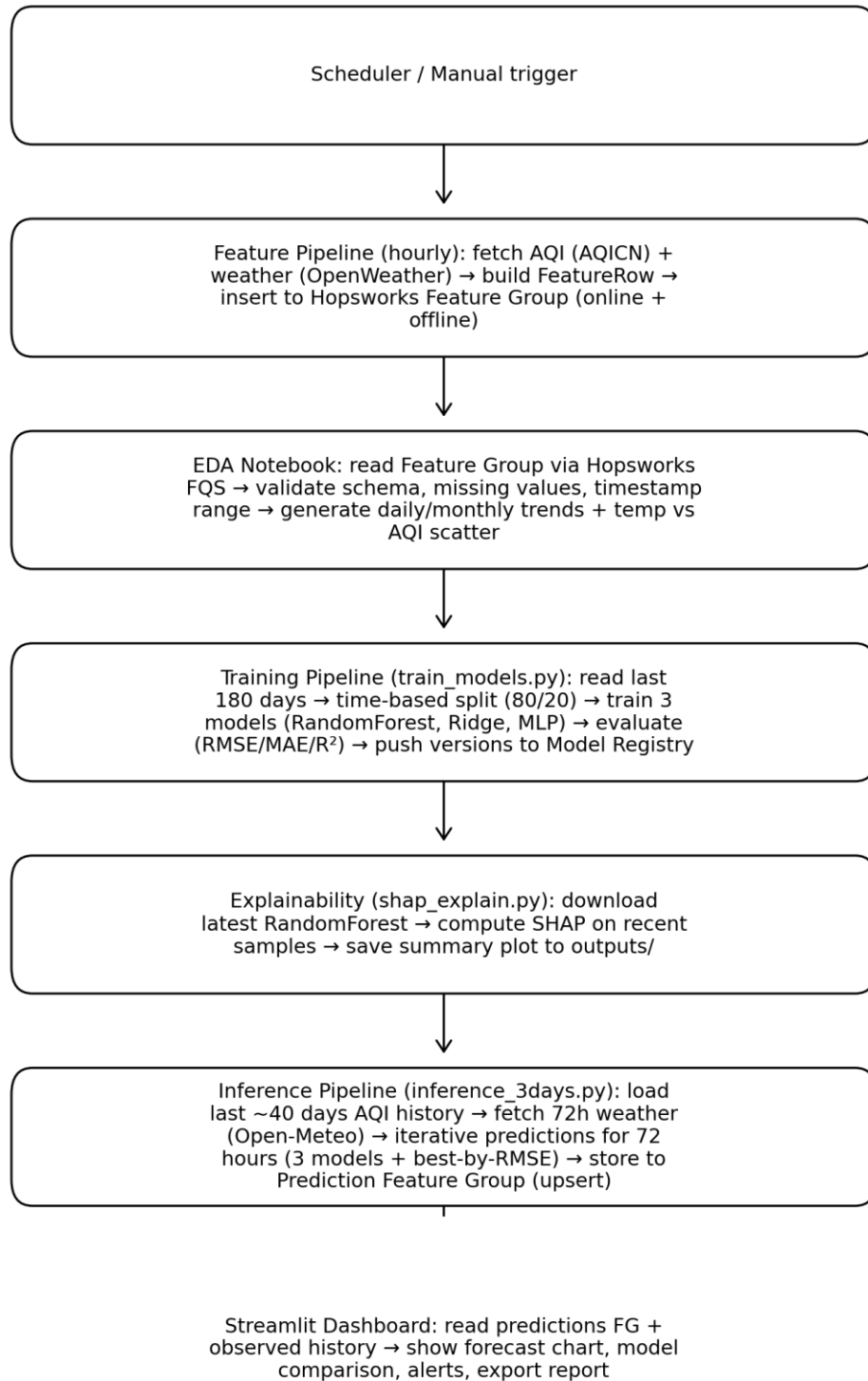


Figure 1 — End-to-end flow from ingestion to dashboard.

High-Level Architecture (Components)

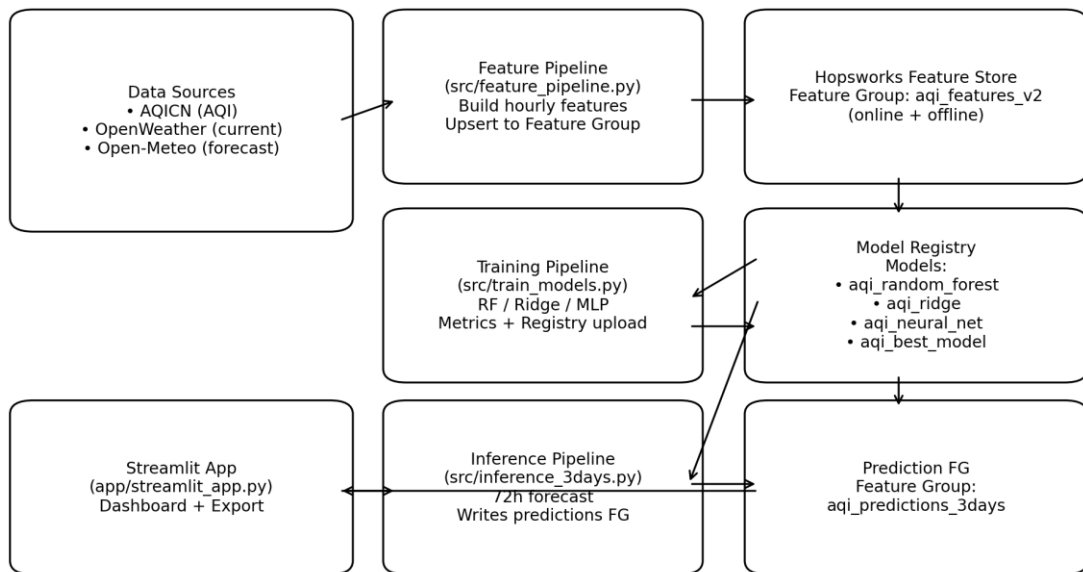


Figure 2 — Component-level architecture (data sources, Hopsworks, pipelines, dashboard).

USE CASE DIAGRAM

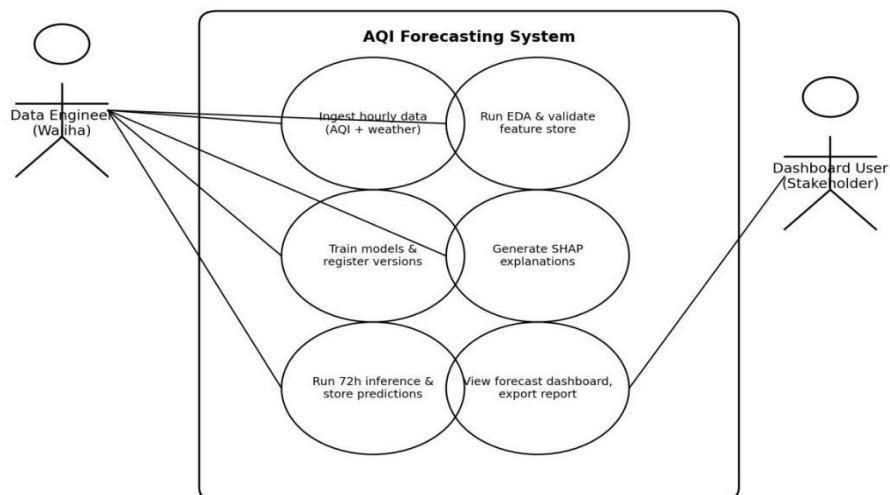


Figure 3 — Use-case view (engineering workflow and stakeholder dashboard usage).

6. Data Ingestion Pipeline (Feature Store)

The first pipeline operates on an hourly cycle and is tasked with creating the training-quality dataset. It retrieves the latest AQI values for Islamabad as well as the prevailing weather, and it translates these into a single structured feature row. One of

the most important architectural choices made was to retain one row per hour, considering this to be the inherent resolution of the system.

This pipeline is designed to ensure that the data, no matter how inconsistent it may be (for example, timezone formatting), is stored in a manner that is conducive to both offline model training and online prediction.

6.1 Feature store schema

The Feature Group **aqi_features_v2** stores the complete feature set, including:

- Timestamp in UTC (**timestamp_utc**)
- A string-based primary key (**event_time**)
- Location identifier (**city**)
- Time-based features (**hour, day, month, day_of_week**)
- Weather features (**temp_c, humidity, pressure, wind_speed**)
- Target variable (**aqi**)

6.2 Issue faced & fix

- Primary Key Constraint:
 - The Online Feature Store in Hopsworks does not allow timestamp columns in the primary key. To overcome this, I created the event_time column as a string in UTC format and defined the composite key:
 - `primary_key = ['city', 'event_time']`, while maintaining the timestamp_utc column for offline processing.
- Timestamp Format Mismatch:
 - The AQICN timestamps occasionally came in without any timezone details. I standardized all incoming timestamps by parsing them uniformly and coercing them to UTC, to avoid having values with mixed offsets in the data.

7. Exploratory Data Analysis (EDA)

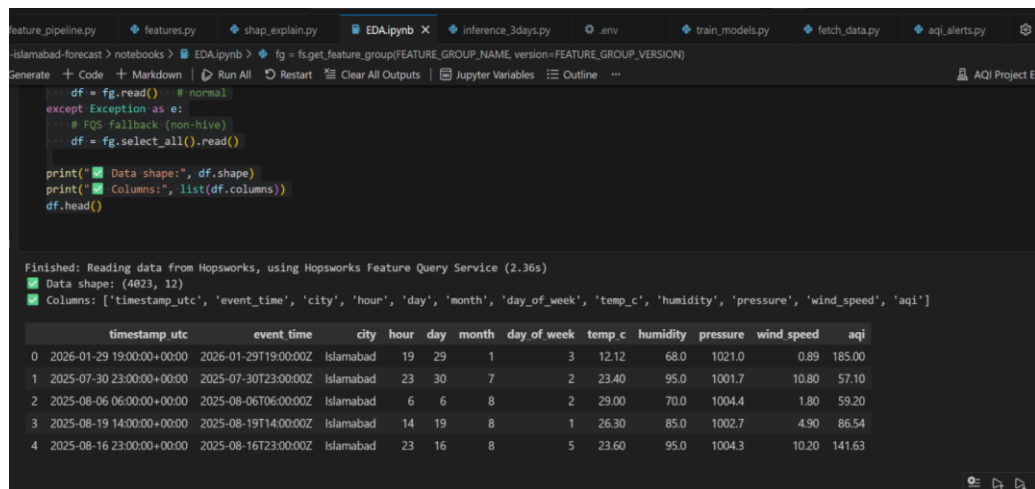
The Exploratory Data Analysis (EDA) was done directly on the data from the Hopsworks Feature Store to ensure that the analysis is based on the same dataset that will be used later for training and inference.

The key areas of focus during the Exploratory Data Analysis were:

- Validation of the Feature Store data by checking the timestamp min/max values
- Identification and removal of duplicate hourly rows
- Analysis of AQI trends and seasonality patterns over time
- Analysis of correlations between weather variables and AQI values

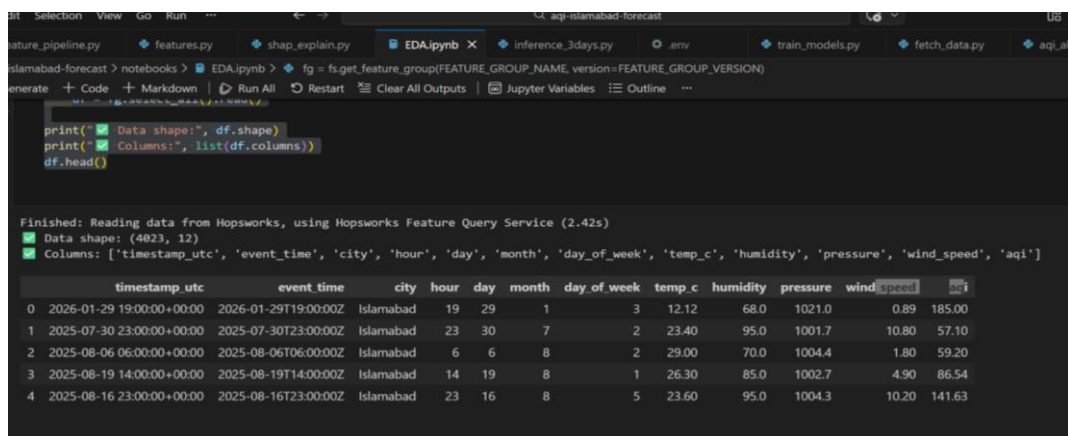
7.1 Reading from Hopsworks (validation)

The first thing I did was check the shape of the data, the columns, and the range of the timestamp. This was important because, in the past, I had issues where the dashboard would display old predictions just because the range of the data stored was not what I expected.



```
feature_pipeline.py | features.py | shap_explain.py | EDA.ipynb X | inference_3days.py | .env | train_models.py | fetch_data.py | aqi_alerts.py |  
Islamabad-forecast > notebooks > EDA.ipynb > fg = fs.get_feature_group(FEATURE_GROUP_NAME, version=FEATURE_GROUP_VERSION)  
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline |  
AQI Project E  
  
df = fg.read() # normal  
except Exception as e:  
    # FQS fallback (non-hive)  
    df = fg.select_all().read()  
  
print("Data shape:", df.shape)  
print("Columns:", list(df.columns))  
df.head()  
  
Finished: Reading data from Hopsworks, using Hopsworks Feature Query Service (2.36s)  
Data shape: (4023, 12)  
Columns: ['timestamp_utc', 'event_time', 'city', 'hour', 'day', 'month', 'day_of_week', 'temp_c', 'humidity', 'pressure', 'wind_speed', 'aqi']  
  
timestamp_utc    event_time    city    hour    day    month    day_of_week    temp_c    humidity    pressure    wind_speed    aqi  
0  2026-01-29 19:00:00+00:00    2026-01-29T19:00:00Z    Islamabad    19    29    1    3    12.12    68.0    1021.0    0.89    185.00  
1  2025-07-30 23:00:00+00:00    2025-07-30T23:00:00Z    Islamabad    23    30    7    2    23.40    95.0    1001.7    10.80    57.10  
2  2025-08-06 06:00:00+00:00    2025-08-06T06:00:00Z    Islamabad    6    6    8    2    29.00    70.0    1004.4    1.80    59.20  
3  2025-08-19 14:00:00+00:00    2025-08-19T14:00:00Z    Islamabad    14    19    8    1    26.30    85.0    1002.7    4.90    86.54  
4  2025-08-16 23:00:00+00:00    2025-08-16T23:00:00Z    Islamabad    23    16    8    5    23.60    95.0    1004.3    10.20    141.63
```

Figure 4 — EDA notebook: reading feature group, schema validation, and timestamp range checks.



```
feature_pipeline.py | features.py | shap_explain.py | EDA.ipynb X | inference_3days.py | .env | train_models.py | fetch_data.py | aqi_al  
Islamabad-forecast > notebooks > EDA.ipynb > fg = fs.get_feature_group(FEATURE_GROUP_NAME, version=FEATURE_GROUP_VERSION)  
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline |  
AQI Project E  
  
print("Data shape:", df.shape)  
print("Columns:", list(df.columns))  
df.head()  
  
Finished: Reading data from Hopsworks, using Hopsworks Feature Query Service (2.42s)  
Data shape: (4023, 12)  
Columns: ['timestamp_utc', 'event_time', 'city', 'hour', 'day', 'month', 'day_of_week', 'temp_c', 'humidity', 'pressure', 'wind_speed', 'aqi']  
  
timestamp_utc    event_time    city    hour    day    month    day_of_week    temp_c    humidity    pressure    wind_speed    aqi  
0  2026-01-29 19:00:00+00:00    2026-01-29T19:00:00Z    Islamabad    19    29    1    3    12.12    68.0    1021.0    0.89    185.00  
1  2025-07-30 23:00:00+00:00    2025-07-30T23:00:00Z    Islamabad    23    30    7    2    23.40    95.0    1001.7    10.80    57.10  
2  2025-08-06 06:00:00+00:00    2025-08-06T06:00:00Z    Islamabad    6    6    8    2    29.00    70.0    1004.4    1.80    59.20  
3  2025-08-19 14:00:00+00:00    2025-08-19T14:00:00Z    Islamabad    14    19    8    1    26.30    85.0    1002.7    4.90    86.54  
4  2025-08-16 23:00:00+00:00    2025-08-16T23:00:00Z    Islamabad    23    16    8    5    23.60    95.0    1004.3    10.20    141.63
```

Figure 5 — EDA notebook: reading feature group, schema validation, and timestamp range checks.


```
df.describe(include="all")
```

Python

	timestamp_utc	event_time	city	hour	day	month	day_of_week	temp_c	humidity	pressure	wind_speed	aqi
count	4023	4023	4023	4023.000000	4023.000000	4023.000000	4023.000000	4023.000000	4023.000000	4023.000000	4023.000000	4023.000000
unique	NaN	4023	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
top	NaN	2025-07-30T18:00:00Z	Islamabad	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
freq	NaN	1	4023	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
mean	2025-10-28 23:54:35.167785+00:00	NaN	NaN	11.519016	16.006711	8.003231	2.984837	19.882329	68.225702	1011.829580	5.429433	136.455048
min	2025-07-30 18:00:00+00:00	NaN	NaN	0.000000	1.000000	1.000000	0.000000	2.800000	26.000000	996.200000	0.000000	12.500000
25%	2025-09-10 15:30:00+00:00	NaN	NaN	6.000000	8.000000	8.000000	1.000000	13.215000	57.000000	1005.600000	3.100000	92.850000
50%	2025-10-22 13:00:00+00:00	NaN	NaN	12.000000	16.000000	9.000000	3.000000	20.460000	69.000000	1012.700000	5.000000	141.630000
75%	2025-12-23 13:30:00+00:00	NaN	NaN	18.000000	24.000000	10.000000	5.000000	26.000000	81.000000	1017.600000	7.100000	169.850000
max	2026-02-06 16:00:00+00:00	NaN	NaN	23.000000	31.000000	12.000000	6.000000	35.600000	100.000000	1030.100000	28.300000	403.970000
std	NaN	NaN	NaN	6.916751	8.927867	3.656393	1.997143	7.587178	15.886040	6.798885	3.207313	55.131077

Figure 6 — EDA notebook: reading feature group, schema validation, and timestamp range checks.

7.2 Visual EDA (trends + relationships)

After validation, I generated three core plots: (a) daily mean AQI trend, (b) monthly mean AQI trend, and (c) temperature vs AQI scatter. Daily and monthly resampling made it easier to see long-term patterns, while the scatter plot helped me understand variance at different temperatures.

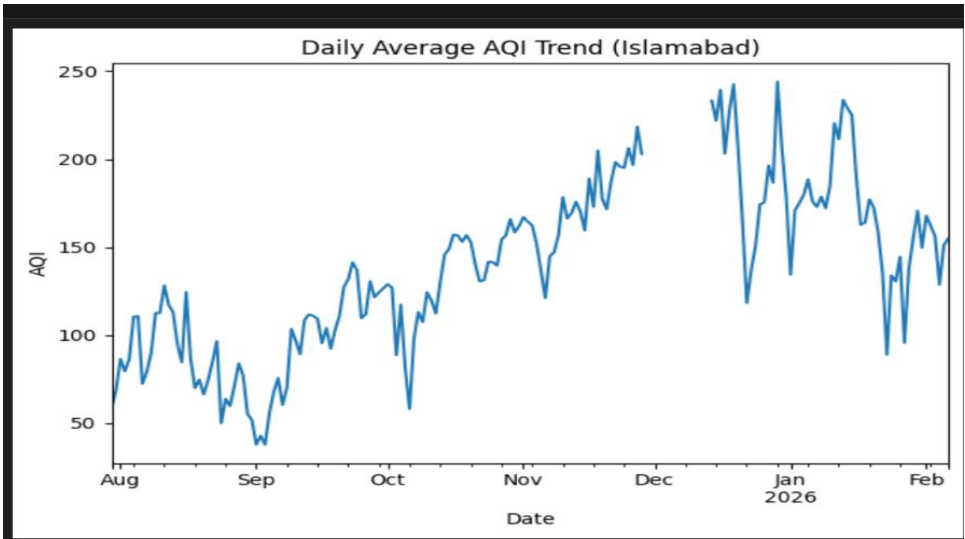


Figure 7 — EDA plots generated from feature store data (trend and relationships).

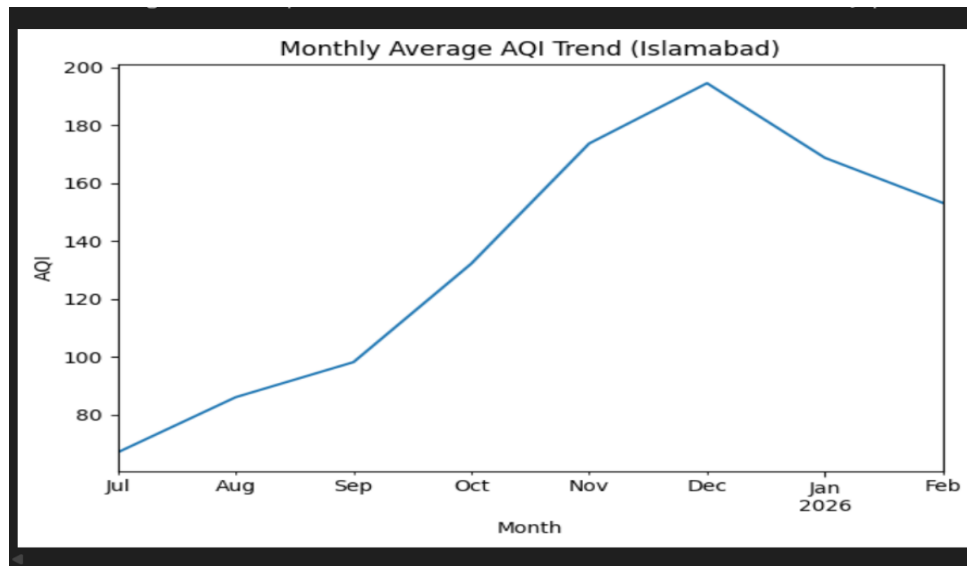


Figure 8 — EDA plots generated from feature store data (trend and relationships).

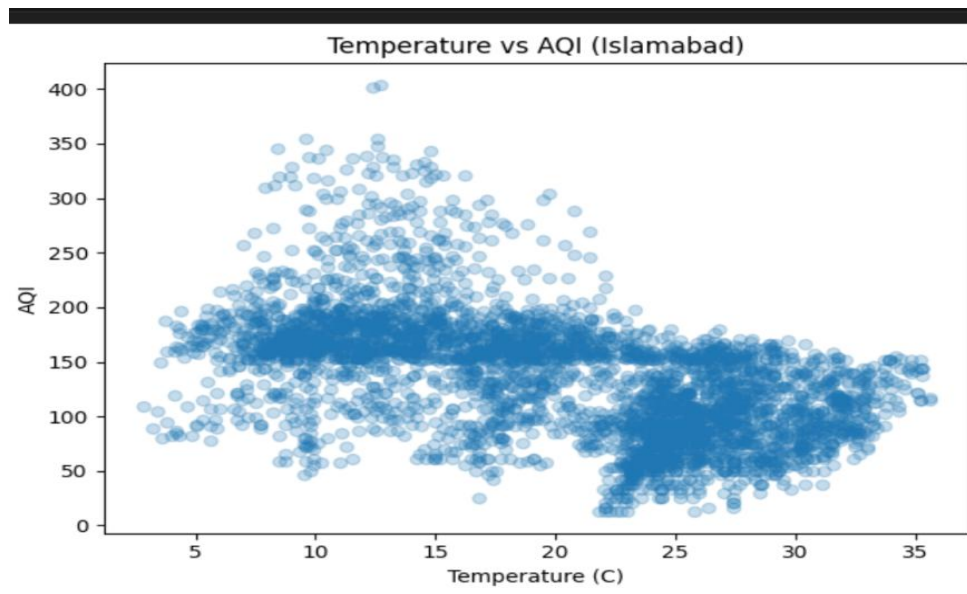


Figure 9 — EDA plots generated from feature store data (trend and relationships).

7.3 What was hardest in EDA (and how it was solved)

The most difficult aspect of the project was not creating the plots but ensuring that the data used to create the plots was accurate. The biggest issue was synchronizing the timestamp (UTC) and removing the duplicate hourly rows to ensure that the resampling and lag features were accurate. After I standardized the `timestamp_utc` to an hourly floor and removed the duplicates, the trend plots were stable, and the engineered variables were accurate.

8. Feature Engineering

The feature engineering was based on the observation that AQI is a time series signal, and the most important predictor of future levels of air quality is often the immediate past. Hence, I divided the feature engineering into two broad categories: features based on calendar and weather variables, and features based on AQI history.

The AQI history features are as follows:

- `aqi_lag1`: AQI one hour ago
- `aqi_lag2`: AQI two hours ago
- `aqi_lag24`: AQI at the same hour of the previous day
- `aqi_roll24`: Rolling mean of the last 24 hours
- `aqi_roll7d`: Rolling mean of the last 7 days
- `aqi_diff1`: Short-term AQI change rate

8.1 Base features

- Time features: `hour`, `day`, `month`, `day_of_week` → capture daily/weekly/seasonal patterns.
- Weather features: temperature, humidity, pressure, wind speed → capture dispersion and meteorological effects.

8.2 Engineered time-series features (no leakage)

First, I always offset the AQI values by one hour, and only then computed the rolling averages.

For instance, `aqi_roll24` is not the mean including the current hour—it's the mean of the previous 24 hours only. This ensures that the forecasting remains realistic, since in real life, you never know the current target value in advance.

The key history-based features that I employed are as follows:

- `aqi_lag1`: AQI from one hour ago
- `aqi_lag2`: AQI from two hours ago (also useful for change rate)
- `aqi_lag24`: AQI at the same hour yesterday (daily pattern)
- `aqi_roll24`: Average AQI over the last 24 hours (shifted)
- `aqi_roll7d`: Weekly rolling average over the last 7 days (168 hours, shifted)
- `aqi_diff1`: The short-term change between last hour and two hours ago

		timestamp_utc	aqi	aqi_lag1	aqi_lag2	aqi_lag24	aqi_roll24	aqi_roll7d	aqi_diff1
78	2025-07-30 18:00:00+00:00	65.51	NaN	NaN	NaN	NaN	NaN	NaN	NaN
66	2025-07-30 19:00:00+00:00	65.51	65.51	NaN	NaN	NaN	NaN	NaN	NaN
152	2025-07-30 20:00:00+00:00	57.10	65.51	65.51	NaN	NaN	NaN	NaN	0.00
265	2025-07-30 21:00:00+00:00	61.30	57.10	65.51	NaN	NaN	NaN	NaN	-8.41
134	2025-07-30 22:00:00+00:00	45.83	61.30	57.10	NaN	NaN	NaN	NaN	4.20
1	2025-07-30 23:00:00+00:00	57.10	45.83	61.30	NaN	NaN	NaN	NaN	-15.47
232	2025-07-31 00:00:00+00:00	37.50	57.10	45.83	NaN	NaN	NaN	NaN	11.27
320	2025-07-31 01:00:00+00:00	50.00	37.50	57.10	NaN	NaN	NaN	NaN	-19.60
71	2025-07-31 02:00:00+00:00	73.92	50.00	37.50	NaN	NaN	NaN	NaN	12.50
465	2025-07-31 03:00:00+00:00	65.51	73.92	50.00	NaN	NaN	NaN	NaN	23.92
471	2025-07-31 04:00:00+00:00	55.00	65.51	73.92	NaN	NaN	NaN	NaN	-8.41
337	2025-07-31 05:00:00+00:00	50.00	55.00	65.51	NaN	NaN	NaN	NaN	-10.51
166	2025-07-31 06:00:00+00:00	61.30	50.00	55.00	NaN	NaN	NaN	NaN	-5.00
225	2025-07-31 07:00:00+00:00	63.41	61.30	50.00	NaN	NaN	NaN	NaN	11.30
172	2025-07-31 08:00:00+00:00	61.30	63.41	61.30	NaN	NaN	NaN	NaN	2.11

Figure 10 — Engineered feature preview (lags, rolling stats, diff).

	timestamp_utc	aqi	alert
4003	2026-02-05 12:00:00+00:00	158.0	⚠️ Unhealthy
4004	2026-02-05 13:00:00+00:00	132.0	😷 Moderate
4005	2026-02-05 15:00:00+00:00	112.0	😷 Moderate
4006	2026-02-05 17:00:00+00:00	157.0	⚠️ Unhealthy
4007	2026-02-05 18:00:00+00:00	175.0	⚠️ Unhealthy
4008	2026-02-05 20:00:00+00:00	199.0	⚠️ Unhealthy
4009	2026-02-05 21:00:00+00:00	215.0	⚠️ Unhealthy
4010	2026-02-05 22:00:00+00:00	179.0	⚠️ Unhealthy
4011	2026-02-06 01:00:00+00:00	170.0	⚠️ Unhealthy
4012	2026-02-06 02:00:00+00:00	159.0	⚠️ Unhealthy
4013	2026-02-06 05:00:00+00:00	161.0	⚠️ Unhealthy
4014	2026-02-06 07:00:00+00:00	162.0	⚠️ Unhealthy
4015	2026-02-06 08:00:00+00:00	161.0	⚠️ Unhealthy
4016	2026-02-06 09:00:00+00:00	127.0	😷 Moderate
4017	2026-02-06 10:00:00+00:00	122.0	😷 Moderate
4018	2026-02-06 11:00:00+00:00	149.0	😷 Moderate
4019	2026-02-06 12:00:00+00:00	162.0	⚠️ Unhealthy
4020	2026-02-06 13:00:00+00:00	160.0	⚠️ Unhealthy
4021	2026-02-06 14:00:00+00:00	162.0	⚠️ Unhealthy
4022	2026-02-06 16:00:00+00:00	163.0	⚠️ Unhealthy

Figure 11 — Alert mapping example based on predicted/observed AQI levels.

9. Model Training Pipeline

Model training in this project is implemented inside the `src/train_models.py` file. The training process retrieves the latest dataset directly from the Hopsworks Feature Store and then trains three different regression models for AQI forecasting.

One of the most important considerations in this project was to implement a time split instead of a random split. The reason for this is that AQI forecasting is a real-time series problem, and the model should always learn from the past data and then forecast the future values.

9.1 Training basis (exact settings used)

For training, I used the most recent 180 days (~6 months) of hourly data available in the feature store. This was chosen as it captures seasonal trends and is still relatively recent.

The specifics are as follows:

- Training window: last 180 days from the newest timestamp
- Max rows: capped at 8000 to prevent runtime issues
- Timestamp cleanup: converted timestamps to UTC, floored to hourly precision, and removed duplicate rows
- Split strategy: strict time split with `TEST_RATIO = 0.20`
- First 80%: training data
- Last 20%: testing data
- Feature set: 14 total features (time, weather, and AQI lag/rolling history features)

9.2 Models trained and why

The pipeline trains three models, each of which was selected for a particular reason:

- **RandomForestRegressor**

This model is very effective on tabular data and is capable of handling complex non-linear relationships between weather features and AQI history. It also makes for a good baseline.

- **Ridge Regression (with StandardScaler)**

Ridge regression is a simple and interpretable linear model. I added it mostly because it gives interpretable coefficients, almost like a forecasting equation.

- **MLPRegressor / Neural Network (with StandardScaler)**

This neural network baseline was trained to check if a deeper non-linear model could do better. Early stopping was turned on to prevent overfitting.

9.3 Evaluation metrics (how results were computed)

To make a fair comparison between the models, I tested each model on the same time-based test set using the following metrics:

- RMSE (Root Mean Squared Error)
- MAE (Mean Absolute Error)
- R^2 Score

The primary metric for selection was RMSE because it gives more weight to large errors, which is particularly significant when the AQI suddenly shoots up.

9.4 Training output (captured run)

The following results are taken from the actual training run log (15 Feb 2026) performed locally within the project virtual environment.

- RandomForest - RMSE: 19.2758 | MAE: 13.9281 | R^2 : 0.8236
- Ridge - RMSE: 19.5086 | MAE: 13.7313 | R^2 : 0.8193
- NeuralNet - RMSE: 21.4984 | MAE: 15.6860 | R^2 : 0.7806

Best model selected: RandomForest (lowest RMSE).

9.5 Ridge formula (interpretation)

Since Ridge is a linear model, it learns a weighted sum of standardized features. To represent the learned relationship, I printed the top terms based on absolute coefficient value. The top terms were the short-term AQI history features (aqi_lag1, aqi_lag2, aqi_diff1, aqi_lag24), which is as expected: the most recent AQI values drive the forecast of the next hour.

- Intercept: 134.5540

Largest coefficients (top terms):

- aqi_lag1 (20.98)
- aqi_lag2 (17.54)
- aqi_diff1 (9.12)
- aqi_lag24 (8.65)

- **aqi_roll24 (7.10)**
- Hour effect was smaller (hour = -1.58)

This matches intuition: the most recent AQI readings dominate the next-hour forecast.

9.6 RandomForest Model Interpretation (Best Performing Model)

Unlike Ridge Regression, RandomForest does not generate a single mathematical equation since it is a combination of many decision trees. Rather, it identifies complex non-linear relationships between AQI historical patterns and meteorological factors.

In this project, RandomForest performed the best in overall forecasting accuracy:

- RMSE: 19.2758
- MAE: 13.9281
- R^2 : 0.8236

This is an indication that AQI patterns in Islamabad are affected by interactions between:

- AQI persistence in the recent past (lag features)
- Rolling trends of pollution
- Weather-driven dispersion factors (wind speed and humidity)

RandomForest was thus chosen as the best model for production purposes since it was able to better capture AQI peaks and non-linear variations compared to linear baselines.

This is also the model that is identified as the Best Model within the Streamlit application.

9.7 Neural Network (MLPRegressor) Interpretation

The third model that was trained in the pipeline was MLPRegressor, which is a neural network model. The reason for including this model was to check if a deeper non-linear model could perform better than tree-based models.

The NeuralNet was trained on:

- standardized feature inputs
- early stopping enabled
- iterative optimization on the time series dataset

However, the NeuralNet performed slightly worse than RandomForest:

- RMSE: 21.4984
- MAE: 15.6860
- R^2 : 0.7806

This indicates that for the current AQI dataset size of approximately 6 months of hourly data, neural networks might need:

- more training data
- more pollutant features
- deeper networks

Hence, NeuralNet was retained as a benchmarking model and not as the best predictor.

9.8 Model Registry versioning

After the training process, the model is stored locally and then pushed to the Hopsworks Model Registry. In addition to the model artifact, I also store the evaluation metrics in the metadata, so that each version is completely reproducible.

In addition, I keep another artifact named `aqi_best_model`, which always refers to the model with the lowest RMSE in the current training run. This is helpful during the deployment step, where inference can automatically load the best model.

10. Explainability (SHAP Pipeline)

To make the forecasting system more understandable, I added a SHAP explanation step for the best model, which was RandomForest. The primary goal of SHAP was to solve a very practical question:

“Which features are actually pushing the AQI prediction up or down?”

Since RandomForest is not an interpretable model like a linear model, SHAP was used to provide a global explanation of the model.

10.1 Why SHAP was useful here

Adding SHAP was useful for several important reasons:

- First, it verified that the model was actually using significant signals, particularly the lagged AQI values and rolling averages, and not just noise.
- Second, it provided a stakeholder-friendly explanation of why the predictions were changing, particularly during AQI spikes, where stakeholders want to know what is causing the pollution levels.
- Finally, it served as a validation mechanism for feature engineering. If some unexpected feature had been the dominant feature, it would have indicated that something was amiss in the feature pipeline.

SHAP Summary Plot Insight

The SHAP summary plot revealed that the most influential factors in making predictions were:

- aqi_lag1
- aqi_roll24
- aqi_diff1

This finding was completely expected, since AQI predictions are highly dependent on the persistence of air quality, which means that the most recent values of air pollution tend to have the most significant impact on future air quality.

11. 72-Hour Inference Pipeline

The workflow of inference is managed within the file ``src/inference_3days.py``. The primary intention of this workflow is to produce a rolling 72-hour AQI forecast starting from the current time, not from the last training cutoff.

This requirement became an important part of the project because, in the previous implementation, the dashboard was only displaying predictions up to a certain older date, making the entire system seem outdated. This problem was resolved by implementing inference in a completely dynamic manner, where every time the script runs, it automatically generates the next 72 hourly timestamps.

11.1 Inference basis (step-by-step)

The pipeline runs in a structured way:

➤ **Load recent AQI history**

It logs into Hopsworks and pulls the latest AQI feature data from the feature store. A seed window of around **40 days** is used so lag and rolling features can be computed properly.

➤ **Load models from the registry**

The latest versions of all three trained models (RandomForest, Ridge, and MLP) are downloaded from the Hopsworks Model Registry, along with their stored RMSE values.

➤ **Best model selection**

The system automatically selects the best model based on the lowest RMSE and stores the reason string so the dashboard can display why that model was chosen.

➤ **Fetch future weather forecast**

A 72-hour weather forecast is pulled from **Open-Meteo**, aligned in UTC, and mapped onto the next 72 hourly timestamps.

➤ **Iterative forward prediction**

For each future hour, the pipeline:

- computes engineered lag/rolling features from the history buffer
- predicts AQI using all three models
- takes the prediction from the best-performing model
- appends that predicted value back into history so forecasting can roll forward hour by hour

This iterative approach is what allows true multi-step forecasting.

Store prediction outputs

Final predictions are saved locally as a CSV for inspection and also upserted into the prediction feature group:

aqi_predictions_3days

11.2 Performance issue faced & fix

To begin with, the inference was slow because the models were downloaded every time the pipeline ran.

- To resolve this issue, I added a local cache system:
- Added a model_cache/ directory
- Cached models based on their name and version

This resolves the issue of the pipeline taking time to run because the models are no longer fetched every time the pipeline is run

11.3 Alerts

To make predictions more actionable, I also introduced a hazardous AQI threshold:

$AQI \geq 200 \rightarrow \text{hazardous_alert} = \text{True}$

This enables the dashboard to automatically flag hazardous air quality intervals without requiring additional processing.

12. Streamlit Dashboard

The design of the dashboard was done with the intention of fast and easy consumption. Rather than just displaying the data, the dashboard displays the AQI forecast in a consumable fashion, has the functionality of refresh/auto-refresh, and has a special section that describes which model was chosen as the best one.

One of the key objectives in this case was to ensure that the “black graph” issue was avoided. This is because, in most cases, the graphs appear cluttered or difficult to interpret.

Dashboard Workflow

The Streamlit application displays predictions directly from the Prediction Feature Group:

- Displays multi-model forecast curves
- Highlights the best-performing model
- Shows AQI health guidance cards for user understanding
- Provides an export/report option for saving results

This ensures that the dashboard is not only interactive but also functional for real-world applications.

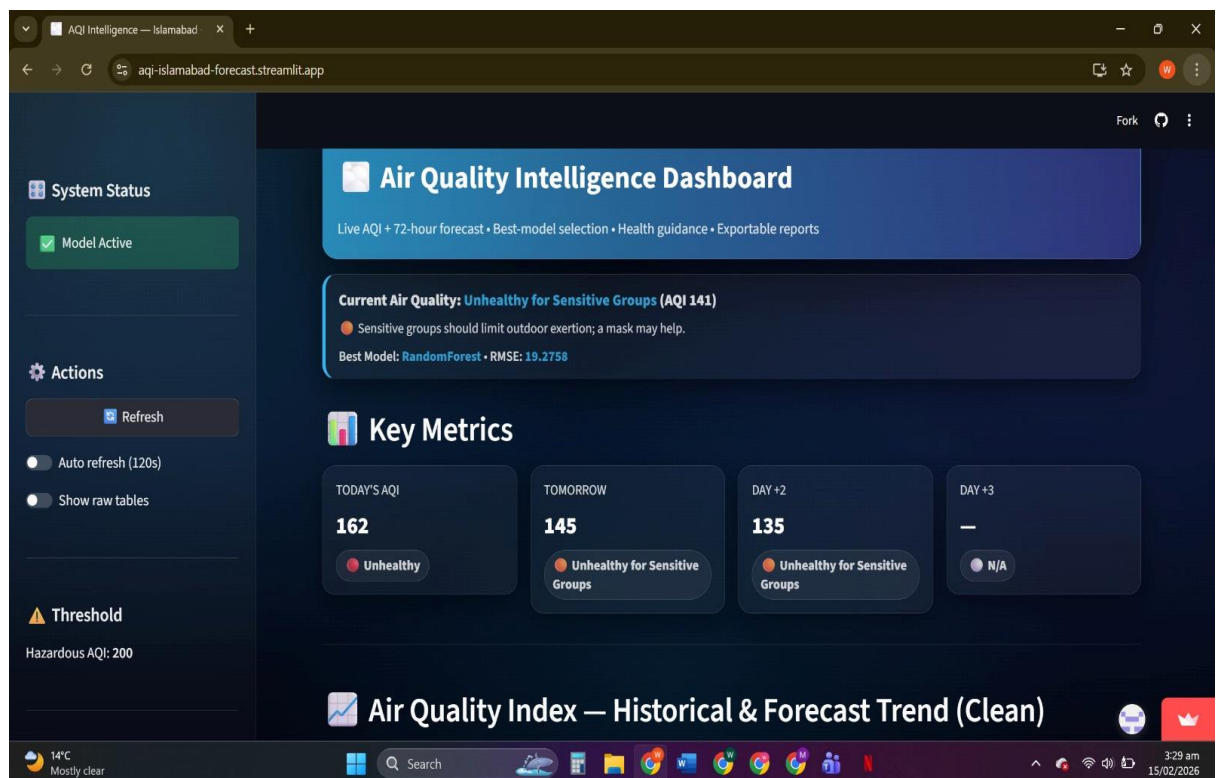


Figure 12 — Dashboard forecast

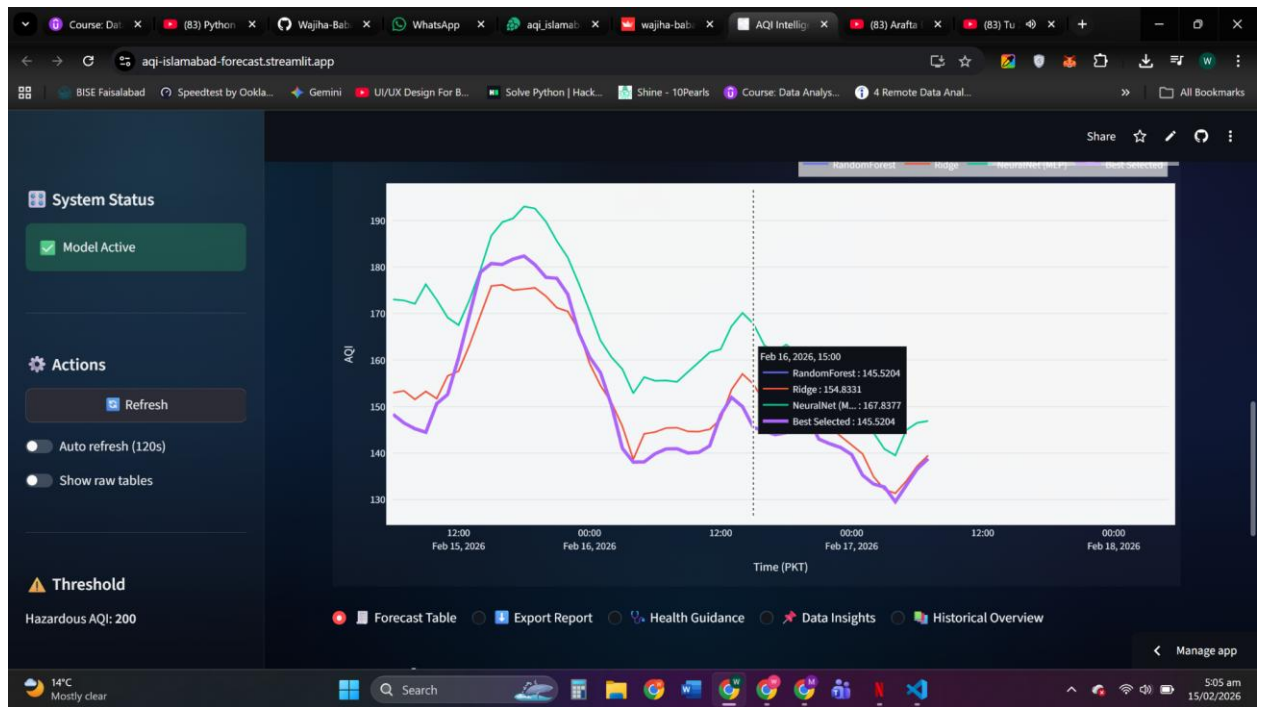


Figure 13 — Dashboard forecast chart with model comparison and best-selected curve.

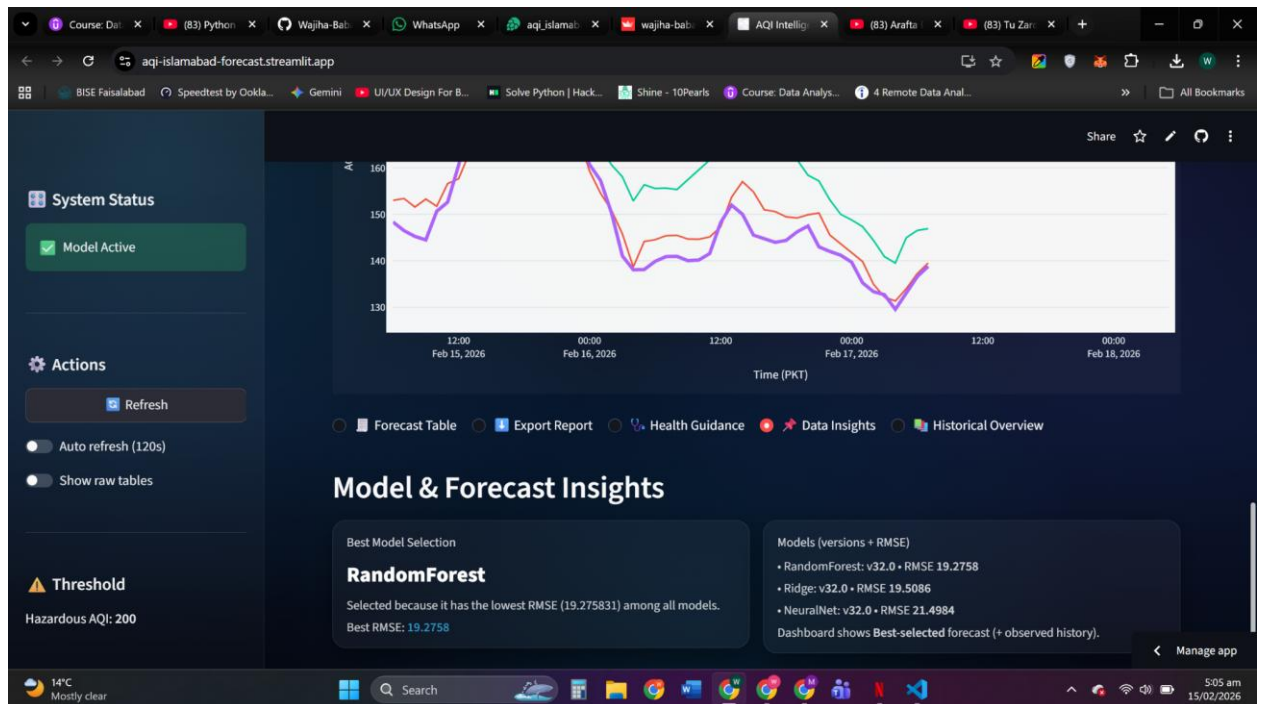


Figure 14 — Model & Forecast Insights section (best model + per-model RMSE).

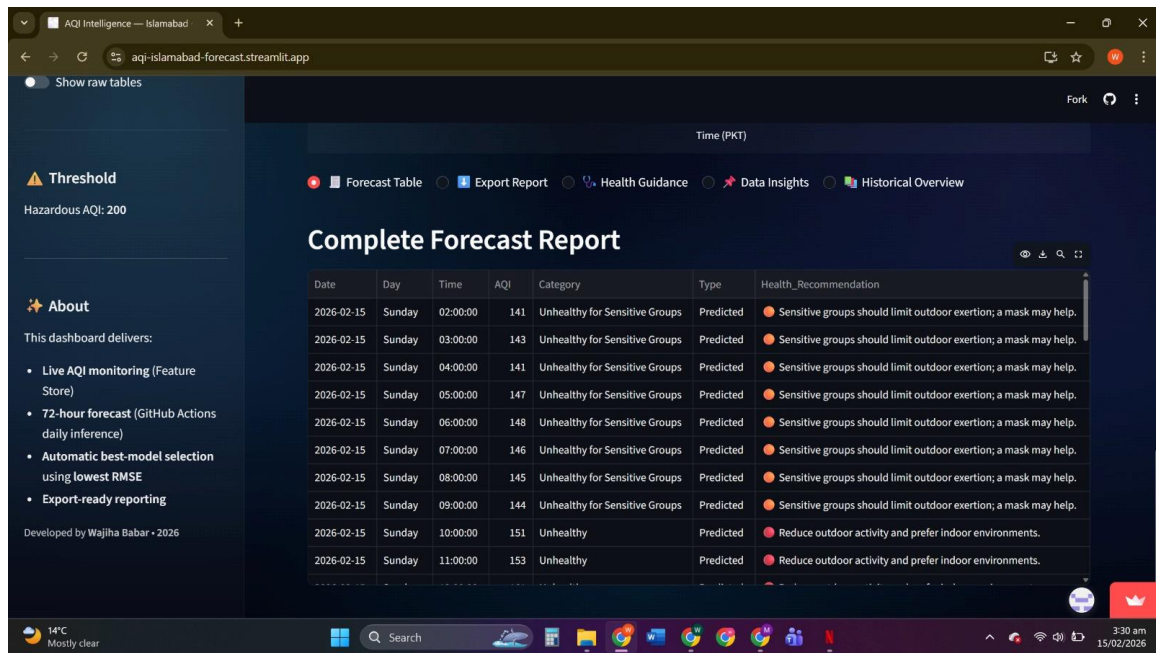


Figure 15 — Complete forecast report.

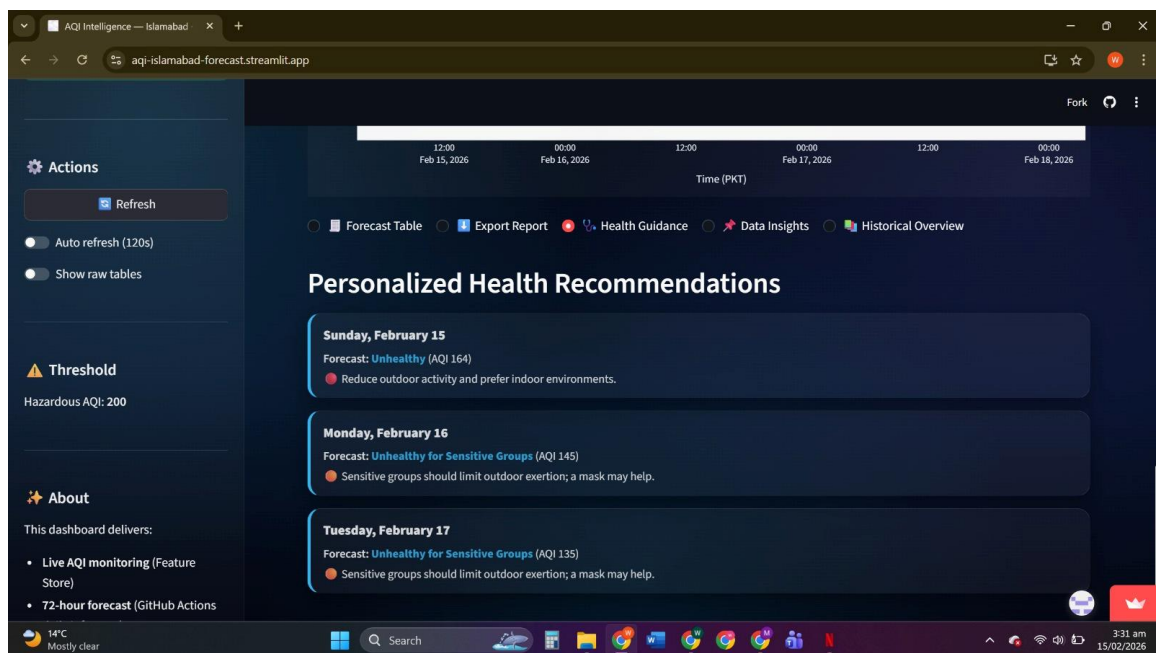


Figure 16— Health Recommendation.

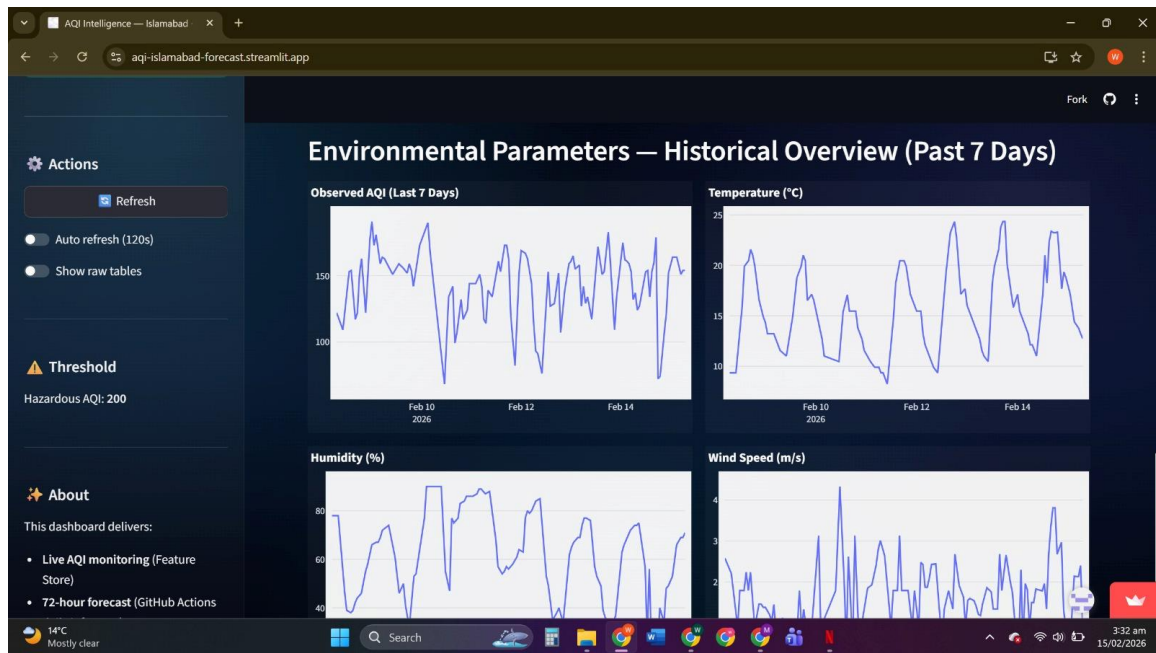


Figure 14 — Enviromantal parameter-Historical overview(past 5 days)

12.1 Dashboard issue noted

While running Streamlit, I did notice some warnings regarding the use of `use_container_width`, which is a deprecated feature. This is not a bug but a reminder to upgrade the code.

Streamlit recommends replacing `use_container_width` with:

`width="stretch"`

or

`width="content"`

13. GitHub Workflow & CI/CD Automation

I continued to maintain the project using a standard Git workflow, which included developing the project locally in a Python virtual environment, making updates to the pipeline in a series of logical commits, and then pushing the changes to the main branch.

To make the system more production-like, I also implemented a GitHub Actions CI/CD workflow, which allows the pipelines to run automatically without having to manually trigger them.

The CI/CD system ensures that the forecasting system remains active and updated through the use of scheduled jobs.

13.1 CI/CD Workflow (GitHub Actions)

Two main automated workflows were maintained:

- **Hourly Feature Ingestion Pipeline**
Runs on a schedule to fetch the latest AQI + weather data and insert new rows into the feature store.
- **Daily / Scheduled Inference Pipeline (72-hour Forecast)**
Runs automatically to generate the next 3-day AQI forecast and update the prediction feature group.

This automation makes the project a “living system” rather than a one-time offline notebook.

13.2 Issues faced in Git workflow

During development, I faced a few practical Git-related challenges:

- **Untracked model artifacts**
The folder `model_cache/` and large trained model files were excluded using `.gitignore` to prevent bloating the repository.
- **Frequent iteration on scripts**
Files like `streamlit_app.py`, `train_models.py`, and `inference_3days.py` required repeated updates, so I had to commit carefully to keep the Git history clean and readable.

14. Testing & Validation

- **Data validation:** checked schema, timestamp range, duplicates, and missing values before training.
- **Training validation:** time-based split ensured evaluation mimics real forecasting usage.
- **Inference validation:** verified that 72 rows are produced and written to both CSV and prediction feature group.
- **Dashboard validation:** confirmed the best model label and RMSE values match the registry metrics used in inference.

15. Conclusion & Next Improvements

This project delivered a complete, versioned, and repeatable AQI forecasting workflow. The strongest model for the current dataset was RandomForest, and the system now supports automated 72-hour forecasts driven by live weather inputs.

Next improvements

- Add a scheduled retraining job (e.g., weekly) to keep models updated as seasons change.
- Create a Hopsworks Feature View for stronger provenance tracking and to remove the provenance warning.
- Add monitoring: drift checks on AQI distribution and alert if data ingestion stops.
- Refine UI: handle screen-size issues and keep consistent theme across charts and tables.